

Name: _____
Date: _____

Student Activity Sheet (SAS)

Course Name: Information Assurance and Security 2

Module Number: 06

Data Security Implementation with Hashing and Encryption

Objectives:

After completing this lab, the student will be able to:

1. Implement password hashing using bcrypt
2. Create secure file encryption using AES-256-CBC
3. Apply Role-Based Access Control (RBAC) principles
4. Test security implementations against common attacks

Prerequisites

- Basic Python programming knowledge
- VS Code or any Python IDE installed
- Internet connection for research

Deliverables Checklist

Submit the following files in notepad and screenshot in a ZIP archive then upload it in gcr (only the leader is required to upload):

- Activity6_setup_check.py (with your screenshot)
- password_manager.py (completed implementation)
- test_password_manager.py (with test results)
- file_encryptor.py (completed implementation)
- test_file_encryption.py (with test results)
- rbac_system.py (completed implementation)
- test_rbac.py (with test results)
- security_tests.py (with test results)
- Activity6_report.docx (do the activity and answer the questions below)



Part 1: Environment Setup

Task 1: Setup Verification

Create a Python script to verify your environment:

```
# Activity6_setup_check.py
import sys
import os

def check_environment():
    print("=" * 60)
    print("Activity 6: ENVIRONMENT CHECK")
    print("=" * 60)

    # Check Python version
    print(f"✓ Python Version: {sys.version}")

    # Check current directory
    print(f"✓ Current Directory: {os.getcwd()}")

    # Check for required modules
    required_modules = ['hashlib', 'os', 'json', 'base64']
    print("\nChecking required modules:")
    for module in required_modules:
        try:
            __import__(module)
            print(f" ✓ {module}")
        except ImportError:
            print(f" X {module} - MISSING")

    # Optional advanced modules
    print("\nOptional modules (for bonus tasks):")
    optional_modules = ['Crypto', 'bcrypt', 'getpass']
    for module in optional_modules:
        try:
            __import__(module)
            print(f" ✓ {module} - AVAILABLE")
        except ImportError:
            print(f" X {module} - Not installed (ok for basic tasks)")
```



```
print("\n" + "=" * 60)
print("ENVIRONMENT CHECK COMPLETE")
print("=" * 60)
```

```
if __name__ == "__main__":
    check_environment()
```

Instructions:

1. Copy the code above
 2. Save as Activity6_setup_check.py
 3. Run it using: python lab6_setup_check.py
 4. Take a screenshot of the output
-

Part 2: Password Hashing Implementation**Task 2.1: Create Password Manager Class**

Implement a password manager that securely stores user credentials:

password_manager.py - STUDENT TO COMPLETE

```
import hashlib
import os
import json
import secrets
```

```
class PasswordManager:
    def __init__(self, storage_file='passwords.json'):
        self.storage_file = storage_file
        self.users = self._load_users()

    def _load_users(self):
        """Load users from storage file"""
        # TODO: Complete this method
        # Hint: Use json.load() and handle FileNotFoundError
        pass

    def _save_users(self):
```



```
"""Save users to storage file"""
```

```
# TODO: Complete this method
```

```
# Hint: Use json.dump() with indent=4
```

```
pass
```

```
def hash_password(self, password, salt=None):
```

```
    """
```

```
    Hash a password using SHA-256 with salt
```

```
    Returns: (hashed_password, salt_used)
```

```
    """
```

```
    if salt is None:
```

```
        salt = secrets.token_hex(16) # Generate random salt
```

```
# TODO: Complete this method
```

```
# Hint: Use hashlib.sha256()
```

```
# Combine salt + password, then hash
```

```
pass
```

```
def verify_password(self, password, stored_hash, salt):
```

```
    """Verify if password matches stored hash"""
```

```
# TODO: Complete this method
```

```
# Hint: Hash the input password with the same salt
```

```
# Compare with stored_hash
```

```
pass
```

```
def add_user(self, username, password, role='user'):
```

```
    """Add a new user with hashed password"""
```

```
# TODO: Complete this method
```

```
# 1. Check if username already exists
```

```
# 2. Hash the password with a new salt
```

```
# 3. Store username, hashed password, salt, and role
```

```
# 4. Save to file
```

```
pass
```

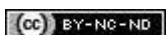
```
def authenticate(self, username, password):
```

```
    """Authenticate a user"""
```

```
# TODO: Complete this method
```

```
# 1. Find user by username
```

```
# 2. Verify password
```



3. Return (success, role) or (False, None)

pass

```
def change_password(self, username, old_password, new_password):
```

```
    """Change user password"""
```

```
    # TODO: Complete this method
```

```
    # 1. Verify old password
```

```
    # 2. Hash new password with new salt
```

```
    # 3. Update stored credentials
```

```
    pass
```

Task 2.2: Test Your Implementation

Create a test script:

```
# test_password_manager.py
```

```
from password_manager import PasswordManager
```

```
def test_password_manager():
```

```
    pm = PasswordManager('test_passwords.json')
```

```
    # Test 1: Add user
```

```
    print("Test 1: Adding user 'alice'")
```

```
    pm.add_user('alice', 'Password123!', 'admin')
```

```
    # Test 2: Authentication - correct password
```

```
    print("\nTest 2: Authenticating with correct password")
```

```
    success, role = pm.authenticate('alice', 'Password123!')
```

```
    print(f" Success: {success}, Role: {role}")
```

```
    # Test 3: Authentication - wrong password
```

```
    print("\nTest 3: Authenticating with wrong password")
```

```
    success, role = pm.authenticate('alice', 'WrongPassword')
```

```
    print(f" Success: {success}, Role: {role}")
```

```
    # Test 4: Change password
```

```
    print("\nTest 4: Changing password")
```

```
    pm.change_password('alice', 'Password123!', 'NewPassword456!')
```

```
    # Test 5: Verify new password works
```

```
    print("\nTest 5: Verifying new password")
```



```
success, role = pm.authenticate('alice', 'NewPassword456!')
print(f" Success: {success}, Role: {role}")

# Test 6: Old password should not work
print("\nTest 6: Old password should fail")
success, role = pm.authenticate('alice', 'Password123!')
print(f" Success: {success} (should be False)")

if __name__ == "__main__":
    test_password_manager()
```

Part 3: File Encryption System

Task 3.1: Implement AES File Encryption

Create a secure file encryption system:

file_encryptor.py - STUDENT TO COMPLETE

```
import os
import hashlib
from base64 import b64encode, b64decode

class FileEncryptor:
    def __init__(self, key):
        """
        Initialize with encryption key
        key: String password (will be hashed to 32 bytes)
        """
        # TODO: Convert key string to 32-byte key using SHA-256
        pass

    def _xor_encrypt(self, data, key):
        """Simple XOR encryption (for learning purposes)"""
        # TODO: Implement XOR encryption
        # Hint: For each byte in data, XOR with key byte
        # Use modulo to cycle through key bytes
        pass
```



```
def _pad_data(self, data, block_size=16):
    """Add PKCS7 padding"""
    # TODO: Implement PKCS7 padding
    # Calculate padding_length = block_size - (len(data) % block_size)
    # Create padding bytes where each byte = padding_length
    # Return data + padding
    pass

def _unpad_data(self, data):
    """Remove PKCS7 padding"""
    # TODO: Implement PKCS7 unpadding
    # Get padding_length from last byte
    # Validate padding (all padding bytes should equal padding_length)
    # Return data without padding
    pass

def encrypt_file(self, input_path, output_path):
    """Encrypt a file"""
    try:
        # TODO: Complete encryption
        # 1. Read input file
        # 2. Pad data to multiple of block size
        # 3. Encrypt using _xor_encrypt
        # 4. Write to output file
        # 5. Return success status
        pass
    except Exception as e:
        print(f"Encryption error: {e}")
        return False

def decrypt_file(self, input_path, output_path):
    """Decrypt a file"""
    try:
        # TODO: Complete decryption
        # 1. Read encrypted file
        # 2. Decrypt using _xor_encrypt (XOR is symmetric)
        # 3. Remove padding using _unpad_data
        # 4. Write to output file
        # 5. Return success status
```

```
pass
except Exception as e:
    print(f"Decryption error: {e}")
    return False

def create_test_file(self, content, filename):
    """Create a test file with given content"""
    # TODO: Create a file with the given content
    pass
```

Task 3.2: Test File Encryption

Create test scenarios:

```
# test_file_encryption.py
from file_encryptor import FileEncryptor

def test_encryption():
    # Create encryptor with password
    encryptor = FileEncryptor("MySecretPassword123")

    # Test 1: Create and encrypt a file
    print("Test 1: Creating and encrypting test file")
    test_content = "This is a secret message!\nLine 2\nLine 3"
    encryptor.create_test_file(test_content, "test_secret.txt")

    # Encrypt the file
    encryptor.encrypt_file("test_secret.txt", "encrypted.bin")
    print(" ✓ File encrypted: encrypted.bin")

    # Test 2: Decrypt with correct password
    print("\nTest 2: Decrypting with correct password")
    encryptor2 = FileEncryptor("MySecretPassword123")
    success = encryptor2.decrypt_file("encrypted.bin", "decrypted.txt")
    print(f" ✓ Decryption successful: {success}")

    # Test 3: Verify content matches
    print("\nTest 3: Verifying decrypted content")
    with open("decrypted.txt", "r") as f:
        decrypted_content = f.read()
```




```
print(f" Original matches decrypted: {test_content == decrypted_content}")

# Test 4: Try wrong password (should fail)
print("\nTest 4: Testing wrong password")
wrong_encryptor = FileEncryptor("WrongPassword")
try:
    wrong_encryptor.decrypt_file("encrypted.bin", "wrong_decrypt.txt")
    print(" X WRONG: Decryption should have failed!")
except:
    print(" ✓ CORRECT: Decryption failed with wrong password")

if __name__ == "__main__":
    test_encryption()
```

Part 4: RBAC Implementation (20 minutes)

Task 4.1: Implement Role-Based Access Control

Create a simple RBAC system:

rbac_system.py - STUDENT TO COMPLETE

```
import json

class RBACSystem:
    def __init__(self, policy_file='rbac_policy.json'):
        self.policy_file = policy_file
        self.roles = self._load_roles()
        self.users = {}

    def _load_roles(self):
        """Load roles and permissions from policy file"""
        # Default roles if file doesn't exist
        default_roles = {
            'guest': ['read_public'],
            'user': ['read_public', 'write_own', 'read_own'],
            'editor': ['read_public', 'write_own', 'read_own', 'edit_public'],
            'admin': ['read_public', 'write_own', 'read_own',
                     'edit_public', 'delete_any', 'manage_users']
        }
        # TODO: Try to load from file, use defaults if file not found
```



pass

```
def add_user(self, user_id, role='user'):
```

```
    """Add a user with specified role"""
```

```
    # TODO: Validate role exists
```

```
    # TODO: Add user to users dictionary
```

```
    pass
```

```
def check_permission(self, user_id, permission):
```

```
    """Check if user has specific permission"""
```

```
    # TODO: Find user's role
```

```
    # TODO: Check if permission exists in role's permissions
```

```
    # TODO: Return True/False
```

```
    pass
```

```
def can_access_file(self, user_id, filename, action='read'):
```

```
    """Check if user can perform action on file"""
```

```
    # Define file types and required permissions
```

```
    file_permissions = {
```

```
        'public.txt': {'read': 'read_public', 'write': 'edit_public'},
```

```
        f'{user_id}_private.txt': {'read': 'read_own', 'write': 'write_own'},
```

```
        'admin_logs.txt': {'read': 'delete_any', 'write': 'manage_users'}
```

```
    }
```

```
    # TODO: Determine required permission for this file/action
```

```
    # TODO: Use check_permission to verify
```

```
    pass
```

```
def list_users(self):
```

```
    """List all users and their roles"""
```

```
    # TODO: Return formatted list of users
```

```
    pass
```

```
# Sample data to test with
```

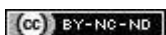
```
sample_users = [
```

```
    ('guest1', 'guest'),
```

```
    ('user1', 'user'),
```

```
    ('editor1', 'editor'),
```

```
    ('admin1', 'admin')]
```



]

Task 4.2: Test RBAC System

Create test cases:

```
# test_rbac.py
from rbac_system import RBACSystem

def test_rbac():
    rbac = RBACSystem()

    # Add test users
    test_users = [
        ('guest1', 'guest'),
        ('user1', 'user'),
        ('editor1', 'editor'),
        ('admin1', 'admin')
    ]

    for user_id, role in test_users:
        rbac.add_user(user_id, role)

    print("Test Cases:")
    print("=" * 60)

    # Test Case 1: Guest permissions
    print("\nTest 1: Guest permissions")
    print(f" Can guest1 read public.txt? {rbac.can_access_file('guest1', 'public.txt', 'read')}")
    print(f" Can guest1 write to public.txt? {rbac.can_access_file('guest1', 'public.txt', 'write')}")

    # Test Case 2: User permissions
    print("\nTest 2: User permissions")
    print(f" Can user1 read own file? {rbac.can_access_file('user1', 'user1_private.txt', 'read')}")
    print(f" Can user1 write to own file? {rbac.can_access_file('user1', 'user1_private.txt', 'write')}")

    # Test Case 3: Admin permissions
    print("\nTest 3: Admin permissions")
    print(f" Can admin1 read admin logs? {rbac.can_access_file('admin1', 'admin_logs.txt', 'read')}")
    print(f" Can admin1 delete any file? {rbac.check_permission('admin1', 'delete_any')}")
```



Test Case 4: Permission escalation attempt

```
print("\nTest Case 4: Attempt permission escalation")
print(f" Can guest1 access admin logs? {rbac.can_access_file('guest1', 'admin_logs.txt', 'read')}")
print(f" Can user1 manage users? {rbac.check_permission('user1', 'manage_users')}")
```

List all users

```
print("\n" + "=" * 60)
print("All Users in System:")
print(rbac.list_users())
```

```
if __name__ == "__main__":
    test_rbac()
```

Part 5: Security Testing and Analysis (10 minutes)

Task 5.1: Vulnerability Testing

Create a security test script:

security_tests.py

```
import os
```

```
def run_security_tests():
    print("SECURITY VULNERABILITY TESTS")
    print("=" * 60)
```

Test 1: Check for plaintext passwords

```
print("\nTest 1: Searching for plaintext passwords in files...")
files_to_check = ['passwords.json', 'test_passwords.json']
for file in files_to_check:
    if os.path.exists(file):
        with open(file, 'r') as f:
            content = f.read()
            if 'password' in content.lower():
                print(f" ⚠ WARNING: '{file}' may contain plaintext passwords")
            else:
                print(f" ✓ '{file}' appears safe")
```

Test 2: Check file permissions



```
print("\nTest 2: Checking file permissions...")
sensitive_files = ['encrypted.bin', 'passwords.json']
for file in sensitive_files:
    if os.path.exists(file):
        # Check if file is readable by others (simplified check)
        try:
            with open(file, 'rb') as f:
                f.read(10) # Try to read a small portion
            print(f" ✓ '{file}' is accessible (normal for lab environment)")
        except PermissionError:
            print(f" ✓ '{file}' has restricted access")
```

Test 3: Validate encryption

```
print("\nTest 3: Validating encryption implementation...")
# Create a test to ensure encrypted files differ from original
if os.path.exists("test_secret.txt") and os.path.exists("encrypted.bin"):
    with open("test_secret.txt", "rb") as f1, open("encrypted.bin", "rb") as f2:
        original = f1.read(100)
        encrypted = f2.read(100)
        if original != encrypted:
            print(" ✓ Encryption is working (files differ)")
        else:
            print(" ❌ CRITICAL: Encryption not working (files identical!)")
```

```
print("\n" + "=" * 60)
print("SECURITY TESTS COMPLETE")
print("=" * 60)
```

```
if __name__ == "__main__":
    run_security_tests()
```

Part 6: Reflection, Think Back, Think About (print this part)**Task 6: Answer the Reflection, Think Back, Think About (PUT HERE THE ANSWER FOR EACH MEMBER)****Reflection:**

- What was the most important security concept you learned from this Activity and how will you apply these security principles in your future projects?

Think Back

Which part of securing data storage was most challenging — hashing or encryption? Why?

Think About

How would you explain the "principle of least privilege" to someone new in IT?

Answer these questions in your based in your activity report:

1. Implementation Challenges:

- What was the most challenging part of implementing password hashing?
- How did you ensure that salts were unique for each password?

2. Security Analysis:



- Why is XOR encryption alone insufficient for real-world applications?
- What additional security measures would be needed for a production system?

3. RBAC Benefits:

- How does RBAC improve security compared to per-user permissions?
- Provide an example where RBAC prevented unauthorized access in your tests.

4. Real-World Application:

- Where have you seen similar security implementations in real applications?
- How would you extend this lab exercise to include multi-factor authentication?

Rubrics:

Component	Points	Criteria	Score
Setup Check	10	Environment verified and screenshot provided	
Password Manager	25	Complete implementation with hashing, salting, and verification	
File Encryption	25	Working encryption/decryption with proper padding	
RBAC System	20	Functional role-based access control	
Security Tests	10	Comprehensive vulnerability testing	
Lab Report	10	Complete answers to all questions	
Total	100		

